CSR 稀疏矩阵向量乘法优化实验报告

2026 年 11 月 1 日

1 背景

本课程设计以开源程序 csrspmv 为研究对象，目标是在保持 y += Ax 语义和原有功能的基础上优化稀疏矩阵向量乘法，并通过实验选择最终交付版本。项目先后完成 v1、v2、v2 系列改进和 v3，最后保留综合验收表现最合适的 v2。本报告介绍原始实现、开发过程、v2 方法与实验依据，并记录最终脚本、README 和上游 PR 交付。

1.1 任务与原始程序

稀疏矩阵向量乘法只处理非零元。CSR 用 values 保存数值、column_indices 保存列号、row_ptr 保存每一行在前两数组中的起止位置。第 i 行读取 row_ptr[i] 到 row_ptr[i+1] 之间的非零元，间接访问 x 的相应元素，将行内乘积累加到已有 y[i]。原项目使用 int64 行偏移和 int32 列号，已避免稠密矩阵中大量零元的存储与计算。[1]

图 1-1 来自原始 master 的真实源码。该单精度路径已包含 OpenMP 行分配和连续 CSR 数据遍历；同一行内的列号决定 x 的间接访问。因此，本项目不能把“采用 CSR”或“使用多线程”作为新增优化贡献。正式原始对照也保留编译器原有自动向量化条件。

Original f32 CSR multiplication and OpenMP worksharing

src/csr.c lines 1278-1300 commit 9352c865

```
1278 static int csr_matrix_int32_spmv_f32_f32_f32(
1279     const struct csr_matrix_int32 * matrix,
1280     const struct vector * src,
1281     struct vector * dst)
1282 {
1283     if (matrix->value_format != csr_value_f32 ||
1284         src->value_format != vector_value_f32 ||
1285         dst->value_format != vector_value_f32)
1286         return EINVAL;
1287     const int64_t * restrict p = matrix->row_ptr;
1288     const int32_t * restrict j = matrix->column_indices;
1289     const float * restrict a = (const float * restrict) matrix->values;
1290     const float * restrict x = (const float * restrict) src->values;
1291     float * restrict y = (float * restrict) dst->values;
1292 #pragma omp for
1293     for (int32_t i = 0; i < matrix->num_rows; i++) {
1294         float z = 0.0;
1295         for (int64_t k = p[i]; k < p[i+1]; k++)
1296             z += a[k] * x[j[k]];
1297         y[i] += z;
1298     }
1299     return 0;
1300 }
```

图 1-1 原始单精度 CSR 计算内核源码摘录

图 1-2 显示原程序把重复调用放在外层 OpenMP 团队中，内核按行分配工作，并在每次调用结束后同步。原始实现并不是每次 SpMV 都重新创建一个团队；后续优化不将原有团队复用当成新增加速。所有版本都真实执行 R 次 y += Ax，不用一次 Ax 的结果替代重复计算。

Original outer OpenMP team and repeated SpMV calls

src/main.c lines 403-411 commit 9352c865

```
403     /* 5. Compute the sparse matrix-vector multiplication. */
404     int repeat = 0;
405     int64_t num_flops = 0;
406 #pragma omp parallel reduction(err_add:err) reduction(max:num_flops) reduction(max:repeat)
407     for (repeat = 0, num_flops = 0; (repeat < args.repeat) || (num_flops < args.min_flops); repeat++) {
408         err = csr_matrix_int32_spmv(&csr_matrix, &x, &y, &num_flops);
409         if (err)
410             break;
411     }
```

图 1-2 原始团队创建与重复调用源码摘录

1.2 优化问题

同样的非零元总数可能对应不同执行成本。短行需要频繁读取行边界和控制循环；集中长行会使等行数分配下的线程工作量失衡；单条巨行不能由普通行划分继续分散；小矩阵的计算量又可能不足以抵消线程与屏障成本。此外，格式转换和完整验证都需要时间，执行阶段更快并不自动意味着用户获得更低总成本。

据此，项目将准备 P、实际 R 次执行 E_R 和清理 D 分别记录，以 C_R = P + E_R + D 作为计算总成本。Matrix Market 解析、COO 转 CSR 等共同输入工作仍记录在进程 wall 中。计算总成本和端到端耗时分别讨论，避免将内核收益误写为程序整体收益。

2 开发过程

2.1 v1 探索与 v2 路线选择

v1 从原始 master 独立开发，比较标量 CSR、打包 CSR 和无排序 SELL8/SELL16。SELL 以切片形式组织若干行，便于跨行 SIMD，但需要填充和派生数组。实验发现其执行时间在部分输入上下降，准备和存储成本却明显增加：在该轮主矩阵 T24/R9 上，SELL16 的准备加执行中位数约为 1.173 s，原始约为 0.789 s，因此未通过主目标验收。该结论促使后续更重视准备成本与强对照，而不是继续扩大格式复杂度。[3]

v2 同样直接从原始 master 创建，与 v1 是并列分支，研究时间上的先后不表示继承了 v1 的代码。v2 保留已有 CSR 数组，以小任务串行、连续工作量分区、固定短行

展开和 merge-path 巨行拆分组合解决不同结构的问题。它没有建立 SELL 副本，主目标 T24/R9 的计算总成本由 0.811400 s 降至 0.734880 s，通过该轮预设验收，成为后续版本的最佳对照。[2]

2.2 v2 系列改进与验收

v2.1 基于 v2 继续研究准备、验证和调度策略，试图扩大受益输入范围。虽然一些新输入得到改善，但主矩阵相对 v2 的总成本中位数增加约 17.48%，且关键条件没有同时通过，因此保留分支和实验记录，工作区回到 v2。

v2.2 继续基于 v2，对适合串行驻留的小任务、少数巨行和高空行比例输入增加针对性策略。它在多类输入中改善了 v2 的不足，但主条件相对 v2 的配对区间跨过 1，未满足预先规定的不退化证据要求。项目没有把“部分场景更好”直接作为替换最终版本的理由。

v2.2.1 从 v2.2 创建，修复抽样混叠导致的空行索引漏选。完整统计后补建索引改善了路径覆盖，但补扫和申请成本也必须计入 P，修复逻辑不等于总成本必然降低。v2.2.2 同样从 v2.2 创建，尝试有界空行区间计划；个别目标出现约 21%的配对总成本改善，但关键收益保留、退化计数和 A/A 精度等共同门槛未全部通过。两个版本均保留记录并回到 v2。

2.3 v3 独立尝试与最终交付

最后一轮 v3 再次从原始 master 创建，采用局部 8 行各 8 项的寄存器转置与跨行 AVX2 gather，不复制整份矩阵。独立校准后冻结默认策略，正式实验包含 104 份输入、1645 个进程及同 FP 机制对照。v3 相对自己的通用后端有明确收益，但主条件相对 v2 的配对时间比为 1.0073，未达到预设至少 5%的改善，关键输入护栏也有失败项，因而没有替代 v2。[4]

版本选择依据每轮提前确定的共同验收标准。后续版本的区间包含 1 时，只能说明证据不足，不能直接称为确定退化或等效。各轮在同批对照内判断，保留失败分支和负面样本，不将不同日期的绝对时间拼成排名。最终核心停留在 optimization/v2，v1 与 v3 的独立路线以及所有 v2 子版本均保留。

2026 年 11 月 1 日的收尾提交命名为 Final optimized version，增加构建、运行、检查脚本并完善 README；算法、Makefile 及单元测试保持已验收 v2 实现。提交

a566fd1 已推送，并向原作者仓库 master 创建 PR #1。PR 当前为 Open、尚未合并；报告不将“已提交”表述为“已被上游接受”。

3 方法

3.1 直接 CSR 计划与安全准备

3.1.1 完整验证和后端选择

v2 的 spmv_plan 只借用原 CSR，保存所选后端、分区方式和必要的线程临时区。prepare 检查维度、类型、行指针单调性与终点，以及全部列号范围；同时统计所有行是否恰好 8 项和最长行长度。行检查与统计融合，大输入的行检查和列检查分别并行执行，两次团队成本均计入 P，不使用免费抽样或按指针地址复用未经验证的旧计划。

仅 f32 矩阵、f32 源向量和 f32 目标向量进入 v2 优化；其他受支持的类型回到原始分派。auto 先依据规模选择串行或并行，再按结构选择 short8、CSR 或 merge；等行数工作量明显不均衡时改用 work 边界。图 3-1 展示真实选择规则及临时区申请，阈值是已冻结的工程规则，不代表任意机器的最优常数。

v2 backend and partition selection

src/spmv_v2.c lines 139-166 commit 9b652e8d

```
139     t=now(); s->all8=all8; s->longest=longest;
140     s->kernel=o->kernel==v2_auto ? (all8 ? v2_short8:v2_csr):o->kernel;
141     if(s->kernel==v2_short8 && !all8) {
142         s->kernel=v2_csr; p->reason="not-all-rows-eight";
143     }
144     int actual=s->serial ? 1:nt;
145     int64_t total=a->num_rows+a->num_nonzeros,max_work=0;
146     for(int t0=0;t0<actual;t0++) {
147         int64_t b=target(a->num_rows,t0,actual),e=target(a->num_rows,t0+1,actual);
148         int64_t w=a->row_ptr[e]-a->row_ptr[b]+e-b;
149         if(w>max_work) max_work=w;
150     }
151     double ideal=(double)total/actual;
152     s->partition=o->partition==v2_part_auto ?
153         (max_work>1.5*ideal ? v2_work:v2_rows):o->partition;
154     if(o->kernel==v2_auto && !o->strict_order && o->partition==v2_part_auto &&
155        actual>1 && longest>2*ideal) s->kernel=v2_merge;
156     if(s->kernel==v2_merge) {
157         s->capacity=actual;
158         if((size_t)actual>SIZE_MAX/sizeof(*s->scratch) ||
159            (size_t)actual*sizeof(*s->scratch)>budget) {
160             s->kernel=v2_csr; p->reason="v2-scratch-budget";
161         } else {
162             s->scratch=malloc((size_t)actual*sizeof(*s->scratch));
163             if(!s->scratch) {s->kernel=v2_csr;p->reason="v2-allocation-failed";}
164             else p->extra_bytes=(size_t)actual*sizeof(*s->scratch);
165         }
166     }
```

图 3-1 v2 内核选择 分区选择与有界临时区源码摘录

3.1.2 生命周期和数值约束

prepare 与 free 在团队外调用，execute 由团队中所有线程一致调用，或走真正的串行入口。矩阵 values、x 和 y 可以逐次变化；行指针、列号、维度或类型改变后必须重建计划。x 与 y 不得重叠，不能让多个团队并发更新同一个 y。负参数、计数溢出及零元矩阵的正 FLOP 目标在调用前统一检查，避免线程在集体同步前出现不同退出路径。

3.2 小任务串行与连续工作量分区

当 auto 模式下的事件量 m + nnz 不超过 32768，或可用线程数为 1 时，v2 在团队外逐次执行原有 y += Ax 语义。它降低小任务的团队和屏障成本，收益属于执行策略选择，不能全部归为单线程内核变快。运行日志同时输出请求配置与实际团队，实验表据此区分同线程比较和自动减少线程的比较。

v2 work cuts and stable merge-path search

src/spmv_v2.c lines 60-88 commit 9b652e8d

```
60  static int64_t target(int64_t total, int t, int nt)
61  {
62      return (total/nt)*t + (t < total%nt ? t : total%nt);
63  }
64  int64_t spmv_v2_row_cut(const struct csr_matrix_int32 *a, int t, int nt, int work)
65  {
66      if (!work) return target(a->num_rows,t,nt);
67      int64_t goal=target(a->num_nonzeros+a->num_rows,t,nt);
68      int64_t lo=0,hi=a->num_rows;
69      while(lo<hi) {
70          int64_t mid=lo+(hi-lo)/2;
71          if(a->row_ptr[mid]+mid<goal) lo=mid+1;
72          else hi=mid;
73      }
74      return lo;
75  }
76  /* Stable tie: row-end events precede equal nonzero-index events. */
77  void spmv_v2_merge_cut(const struct csr_matrix_int32 *a, int64_t diagonal,
78                         int64_t *row, int64_t *nz)
79  {
80      int64_t lo=diagonal>a->num_nonzeros ? diagonal-a->num_nonzeros : 0;
81      int64_t hi=diagonal<a->num_rows ? diagonal:a->num_rows;
82      while(lo<hi) {
83          int64_t mid=lo+(hi-lo)/2;
84          if(a->row_ptr[mid+1] <= diagonal-mid-1) lo=mid+1;
85          else hi=mid;
86      }
87      *row=lo; *nz=diagonal-lo;
88  }
```

图 3-2 v2 工作量边界与稳定 merge 路径搜索源码摘录

对于并行输入，先估计等行数分区的最大工作量。若最大值超过平均值的 1.5 倍，则按 F(i) = row_ptr[i] + i 寻找分界；其中非零元数反映乘加工作，行数反映行处理开销。每个线程对既有 row_ptr 做二分搜索，得到连续行区间，不构建长度为 m 的额外前缀数组。边界查询为每线程 O(log m)，其成本在 E 内支付。

图 3-2 上半部分为 work 边界，下半部分为 merge-path 对角线搜索。work 保持整行归属，在集中长尾中分散非零元，但不能把一条超长行切给多个线程；因此 v2 还需要独立的 merge 后端。空行也对应行事件，能够进入工作量模型并保持真实加零语义。

3.3 固定短行组织与编译设置

完整扫描确认每行恰有 8 项时，short8 把起点写为 8*r，显式执行 8 项乘加，减少通用行循环和边界处理。普通 CSR 路径保留逐项遍历，默认不预取，显式参数可测试 32 或 128 项前瞻。两者都从 sum=0 开始，并在行末只加一次旧 y；空行也执行 y += 0。图 3-3 展示该实现。

v2 ordered direct rows and fixed-eight unrolling

src/spmv_v2.c lines 171-199 commit 9b652e8d

```
171 /* Local ordered scalar operations. No global fast-math/novec. Short8 is a
172  * fixed-width adaptation of Trotter's published unrolling method, not ELL. */
173 #if defined(__GNUC__) && !defined(__clang__)
174 __attribute__((optimize("no-tree-loop-vectorize,no-tree-slp-vectorize")))
175 #endif
176 static void rows_f32(const struct csr_matrix_int32 *a, const float *restrict x,
177                      float *restrict y, int64_t begin, int64_t end, int short8, int d)
178 {
179     const float *restrict v=a->values;
180     const int32_t *restrict j=a->column_indices;
181     int64_t stop=a->row_ptr[end];
182     for(int64_t r=begin;r<end;r++) {
183         int64_t k=short8 ? 8*r:a->row_ptr[r];
184         float sum=0;
185         if(short8) {
186             if(d && d<stop-k) __builtin_prefetch(x+j[k+d],0,1);
187             sum+=v[k]*x[j[k]]; sum+=v[k+1]*x[j[k+1]];
188             sum+=v[k+2]*x[j[k+2]]; sum+=v[k+3]*x[j[k+3]];
189             sum+=v[k+4]*x[j[k+4]]; sum+=v[k+5]*x[j[k+5]];
190             sum+=v[k+6]*x[j[k+6]]; sum+=v[k+7]*x[j[k+7]];
191         } else {
192             for(int64_t e=a->row_ptr[r+1];k<e;k++) {
193                 if(d && d<stop-k) __builtin_prefetch(x+j[k+d],0,1);
194                 sum+=v[k]*x[j[k]];
195             }
196         }
197         y[r]+=sum;
198     }
199 }
```

图 3-3 v2 有序直接 CSR 与固定 8 项展开源码摘录

只有 v2 翻译单元关闭 FMA contraction，行函数局部禁用 GCC 循环与 SLP 向量化；原始 CSR 保留原有编译器能力，项目没有全局关闭自动向量化制造弱基线。short8 是受验证的候选组织方式；同 FP 通用 CSR 对照没有证明展开本身具有普遍稳定的独立优势，报告将收益归于直接 CSR、低准备成本和结构策略的组合。

3.4 超长行拆分与延迟合并

当实际预估团队大于 1、最长行超过平均事件量的两倍，并且允许重组求和时，auto 可以选择 merge。该路径把非零元事件与行结束事件合并成有序序列，按对角线划分事件区间。稳定规则规定相同位置的行结束事件先于非零元事件，坐标均使用 int64，避免将大偏移缩窄到 32 位。该搜索依据原作者 CPU 实现进行 C/f32/int64 适配，来源和 BSD 许可完整保留。[5]

各线程独立处理完整归属行，只把区间两端最多两个不完整行片段写入自己的临时区。所有线程到达屏障后，由单线程按顺序合并边界片段，对每一跨线程行只更新一次旧 y；single 结束的隐式屏障保证后续调用看到完整结果。图 3-4 为延迟合并关键代码。这样拆分巨行时无需对每个乘加执行原子更新。

v2 split-row fragments and delayed update

src/spmv_v2.c lines 207-235 commit 9b652e8d

```
207     spmv_v2_merge_cut(a,target(total,tid,nt),&r,&k);
208     spmv_v2_merge_cut(a,target(total,tid+1,nt),&er,&ek);
209     s->count=0;
210     for(;r<er;r++) {
211         int partial=k>a->row_ptr[r];
212         float sum=0;
213         for(int64_t e=a->row_ptr[r+1];k<e;k++) sum+=v[k]*x[j[k]];
214         if(partial) s->fragment[s->count++]=(struct v2_fragment){r,sum};
215         else y[r]+=sum;
216     }
217     if(k<ek) {
218         float sum=0;
219         for(;k<ek;k++) sum+=v[k]*x[j[k]];
220         s->fragment[s->count++]=(struct v2_fragment){r,sum};
221     }
222 #pragma omp barrier
223 #pragma omp single
224     {
225         int64_t row=-1; float sum=0;
226         for(int t=0;t<nt;t++) for(int f=0;f<slots[t].count;f++) {
227             struct v2_fragment v0=slots[t].fragment[f];
228             if(v0.row!=row) {
229                 if(row>=0) y[row]+=sum;
230                 row=v0.row; sum=0;
231             }
232             sum+=v0.sum;
233         }
234         if(row>=0) y[row]+=sum;
235     } /* single's implicit barrier completes all split-row updates */
```

图 3-4 v2 边界片段与延迟合并源码摘录

每个准备线程的 scratch 为 64 字节，额外空间为 O(T)，而不是 O(nnz)格式副本。预算不足或申请失败回到直接 CSR；实际团队超过准备容量时，全部线程一致走安全行路径。merge 会重组浮点求和顺序，因而使用误差界验证；--strict-order 禁止拆行，但也不承诺跨编译器或不同 FMA 设置的普遍逐位相同。

4 数据

4.1 输入集合与来源

主要结果采用 2026 年 10 月 26 日 v2 验收批次，共 34 份输入：23 份复用输入、2 份独立校准输入、7 份新种子保留输入和 2 份真实矩阵。合成输入覆盖固定与不同短行长度、集中和分散长尾、巨行、空行成簇、小任务及矩形规模；校准与保留集用途分开，保留成绩不反向调整默认规则。

表 4-1 主要输入规模

输入	行数	列数	运行非零元
主矩阵	12,000,000	12,000,000	96,000,000
集中长尾	100,000	12,000,000	1,420,000
单巨行	100,000	2,000,000	1,099,999
保留长尾	50,003	2,000,003	659,009
BCSSTK18	11,948	11,948	149,090
SHERMAN5	3,312	3,312	20,793

主矩阵为 1200 万行与 1200 万列，每行 8 项，共 9600 万非零元；集中长尾与巨行用于检验工作分配，保留长尾使用独立种子。真实矩阵来自 NIST Matrix Market：BCSSTK18 为结构工程对称矩阵，文件存 80519 项，统一对称展开后 149090 项；SHERMAN5 为 3312 阶非对称矩阵，含 20793 项，保留其原始数值分布。[6]

源 MTX、生成器、随机种子、预期结果和 SHA-256 记录在 matrices.json。真实矩阵仅在本地下载和测试，未因原项目采用 GPL 就给矩阵重新声明同一许可，也未将第三方 MTX 直接随 Git 分发。比较时两方使用相同 f32 转换和对称展开参数，避免把输入表示差异当成优化。

4.2 实验环境与公平比较

表 4-2 v2 验收批次环境

项目	记录
处理器	AMD Ryzen 9 7845HX，12 核 24 线程
运行环境	WSL2 Ubuntu 22.04，约 15.26 GiB 内存配置
编译器	GCC 11.4.0，-O3 -march=native
OpenMP	DYNAMIC=FALSE，PLACES=cores
绑定与等待	PROC_BIND=spread，WAIT_POLICY=PASSIVE
主参数	T=1/12/24，R=1/9/100，f32 输入输出

同一批次内原始、v1 和 v2 由固定源码构建，编译器与 ISA 一致，不全局启用 fast-math。原始 csr.c 及链接前 csr.o 的代码段另有哈希核验，但相同对象不保证最终链接布局或系统状态完全相同，因此还设置同二进制诊断和匹配 FP 的通用 CSR 对照。不同后端执行时使用同一矩阵、源向量、初始 y 及相同 R。

v2 验收归档共 1640 个进程，其中 180 个是先导观察，未混入正式结论；正式部分为 1460 个进程。中等输入通常每条件 5 个独立进程；主矩阵每个 T/R 条件含 10 组相邻随机 AB/BA 原始与 v2 进程对。以完整进程对为重采样单位进行 20000 次固定种子 bootstrap，报告加速比中位数的 95%区间。同一进程内 R 次计算相互关联，不当成 R 个独立样本。

总成本对每个进程先计算 P + E_R + D，再对进程结果取中位数；不把三个分项中位数相加冒充总成本中位数。主图使用独立进程总成本中位数之比 S = C_original / C_v2，区间来自配对加速比，二者是不同统计量。原始最佳已测线程为 24，主目标固定 v2 请求 24 线程与之比较，避免只选择弱原始配置。

4.3 正确性与可复现检查

v2 单元测试在 native、portable 及 ASan/UBSan 等配置下验证，每个常规配置包含 5368 项检查。内容覆盖变化 x、非零初始 y、重复调用、空行、非法 CSR、团队变化和注入式分配失败；另有 64 位偏移与统一团队检查。精确二进制分数输入使用精确参考，真实浮点输入使用独立高精度参考及行长相关误差界，不把所有误差都记为零。

本次最终交付额外运行 portable 与 native 构建及单元检查，并用 test.mtx 的独立已知结果核对 original、v2 两种 CLI 在 R=3 时得到向量(9,3,9,6)。还验证 8 组脚本参数、调用目录和后端覆盖行为。运行脚本不会免费预热、追加重复次数或自动重建；构建脚本先清理对象，避免切换编译参数后复用旧产物。

5 结果

本节集中展示 v2 相对原始实现的优势场景，所有图表来自同一 v2 验收批次，并保留具体 T、R 与总成本口径。所选结果用于说明不同策略的有效条件，不据此计算全输入平均收益或宣称任意矩阵都更快；完整逐例结果可由材料仓库复核。

5.1 主矩阵总成本下降

表 5-1 主矩阵 R 等于 9 的总成本

T	原始/s	v2/s	加速比
1	6.728788	5.491452	1.225
12	0.984484	0.870012	1.132
24	0.811400	0.734880	1.104

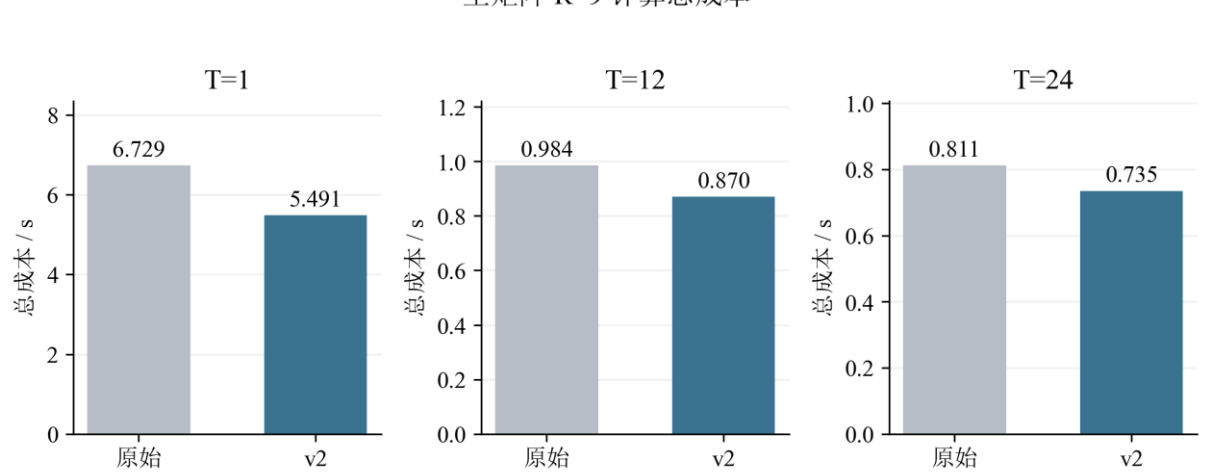


图 5-1 主矩阵各线程计算总成本比较

主目标 T24/R9 中，原始总成本中位数为 0.811400 s，v2 为 0.734880 s，减少 9.43%，总成本中位数之比为 1.104 倍。10 组相邻进程对的加速比中位数 95%区间约为 [1.052,1.211]，支持该条件下的收益。收益包含完整准备和清理后仍成立，超过事先规定的至少 5%主目标门槛。

在相同主矩阵 R=9 条件下，T1 与 T12 的总成本中位数之比分别约 1.225 倍和 1.132 倍。图 5-1 按线程分面显示各自数值，不把串行、物理核与超线程条件混为同一控制；主目标仍以原始最佳已测 24 线程作参照。T24/R100 时，原始 9.305138 s 与 v2 7.780043 s 的总成本之比约 1.196 倍，说明重复使用有助于摊薄准备成本。

5.2 长尾与巨行的负载分配收益

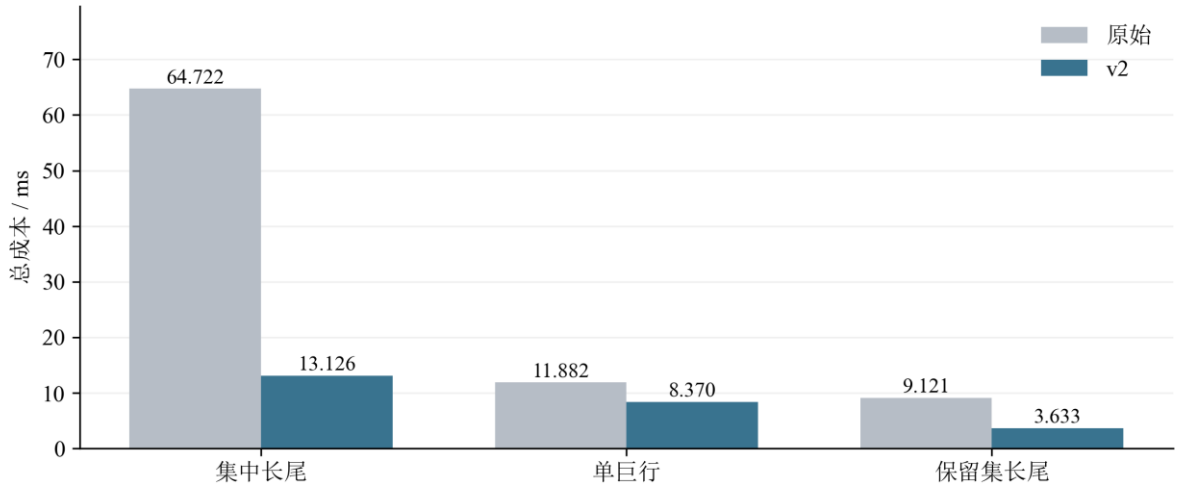


图 5-2 请求 12 线程下不均匀输入的计算总成本

集中长尾 heavy-clustered 在 T12/R9 下，原始总成本 64.722 ms，v2 为 13.126 ms，加速约 4.931 倍。该输入中长行集中，等行数划分会集中非零元；v2 选择 csr/work，以连续事件量边界分散工作。该收益与改变任务分配相符，不能全部归因于短行展开或 SIMD。

单巨行 giant-row 在相同 T12/R9 下由 11.882 ms 降至 8.370 ms，加速约 1.420 倍，v2 实际选择 merge；新种子保留长尾由 9.121 ms 降至 3.633 ms，加速约 2.511 倍。保留输入与开发输入共同展示按结构选择分区和拆行的价值。这些均为包含准备的总成本中位数，普通场景每条件 5 个进程，其描述性比值不冒充额外的显著性检验。

5.3 真实小矩阵的执行策略收益

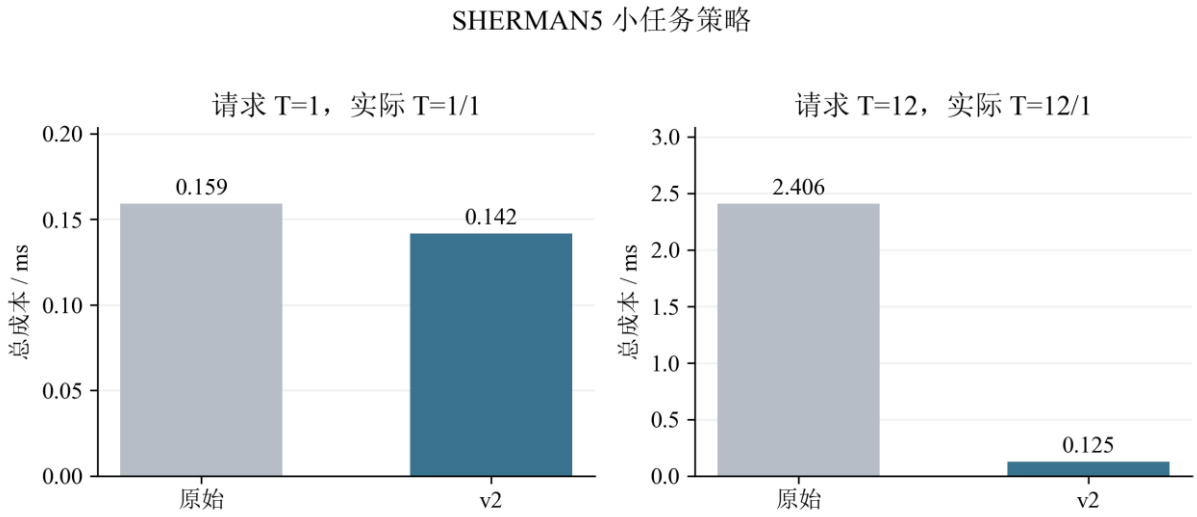


图 5-3 SHERMAN5 同线程与自动串行策略比较

真实矩阵 SHERMAN5 在 T1/R9 下，原始总成本 0.159 ms，v2 为 0.142 ms，总成本比约 1.123 倍。请求 T12 时，原始实际使用 12 线程，总成本 2.406 ms；v2 依据小任务条件实际使用 1 线程，总成本 0.125 ms，描述性加速比约 19.263 倍。图 5-3 明确标注两者实际线程，后一个较大倍数主要体现自动串行规避团队成本，不能称为同线程内核获得 19 倍加速。

上述微秒级输入的绝对差值较小，进程 wall 分辨率与系统活动会影响其解释。报告保留毫秒总成本与请求/实际线程，优势结论限于本批观测；真实输入的存在补充了结构化合成矩阵，仍不能代表所有科学计算矩阵。

6 分析

6.1 优势来源与成本模型

v2 的主要价值是以现有 CSR 为中心，将输入结构、工作分配与执行开销结合起来。集中长尾通过 work 减轻线程工作不均衡，单巨行通过 merge 突破整行归属限制，小任务通过真正串行入口减少同步。主矩阵则受益于低准备成本的直接计算组织。不同场景由不同策略贡献收益，不能用“加了线程”或“采用更宽向量”统一解释。

若执行单次节省 $\Delta t = t_original - t_v2$ 大于 0，则粗略回本条件为 $R > (P + D) / \Delta t$。该模型说明，准备越重，越需要重复使用才能回本；v2 不建立 O(nnz)的派生格式，对

本项目 R=9 的验收条件尤其有利。实际改变 R 还可能影响缓存、频率和同步占比，因此采用独立 R 实验的实测总成本，模型只用于解释，不代替测量。

以主矩阵为例，f32 数值与 int32 列号的逻辑读取量约 8 字节/非零元，9600 万项约 768 MB；再加 int64 行指针以及 x/y 访问，算术强度较低，间接访问和数据移动值得关注。这是逻辑字节模型，不能直接等同于实际 DRAM 流量。项目未锁宿主频率，也未测缓存、持续内存带宽或 Roofline 上限，所以不宣称已经证明达到带宽极限。

6.2 最终选择与复现结论

v2 通过主目标验收，后续各版本则没有在其各自预设条件下建立足够的整体替代证据，因此最终保留 v2。该选择表达项目验收结论，不是所有输入、线程和机器的绝对排名。结果章节展示的优势仍需与明确的输入、编译、等待策略和成本范围一起使用；配对区间未作多重比较校正，也不覆盖跨机器变化。

最终版本保持 original 为可执行文件默认入口，运行脚本显式选择 v2；其他类型保留原始分派，严格顺序模式可禁用拆行。计算数据或结构变化的生命周期要求、完整验证成本、错误检查和许可归属均随代码交付。性能优化与接口兼容、数值验证和可复现脚本共同构成课程成果。

项目已补齐可复用构建/运行/检查脚本及 README，并提交上游 PR。至报告生成时，PR 仍在等待作者审阅，未获得合并结论。最终工程交付保持研究分支、固定源码和原始数据记录，报告中的性能数字能够定位到具体批次及输入，而不依赖重新运行后覆盖旧结果。

6.3 资料与复核入口

[1] 原始 csrspmv 源码与 README

[2] v2 固定批次实验 数据与完整结果

[3] v1 开发与格式转换实验

[4] v3 最终路线实验与验收

[5] Merrill 与 Garland 的 CPU merge 实现

[6] NIST Matrix Market 矩阵集合

[7] v2 方法说明与许可来源

报告数据入口为材料仓库 experiments/2026-10-26-v2 中的 summary.json、paired-analysis.json、matrices.json、source-provenance.json 和 audit.json；图表逐项数据另存于

本报告目录 chart-data.json。源码截图取自固定提交，逐图文件、行号和内容校验和保存在 figure-sources.json。

7 提交记录

7.1 核心仓库提交记录

图 7-1 为 csrspmv 仓库执行 git log --all --author=visiby 后的真实输出截图。--all 覆盖保留的各优化分支，避免只看当前 v2 时漏掉 v1、v3 或已拒绝版本。最上方 a566fd1 为 Final optimized version，其父提交 9b652e8 为已验收 v2；两个提交共同构成当前上游 PR，不合入其他优化路线。



图 7-1 csrspmv 各保留分支的本人提交记录

主要谱系为 master 分别派生 v1、v2、v3；v2.1 和 v2.2 从 v2 派生，v2.2.1 与 v2.2.2 从 v2.2 派生。最终文档和脚本提交加在 v2 上，没有压缩或重写研究历史，原始 master 保持 9352c86。所有版本均能通过原提交与分支复核。

7.2 材料仓库提交记录

图 7-2 为 spmv-course-materials 的实际 main 历史，包含 baseline、方案设计、实验结果和项目日志。后期各优化版本按设计、代码实现、实验记录三个阶段分别提交并推送：设计和实验在材料 main，代码在核心对应分支。截图在本报告归档提交之前采集，承载本报告自身的最后材料提交不做自引用，可由在线历史继续查看。

spmv-course-materials — design and experiment commits

Verbatim Git output • captured 2026-11-01 • no history rewritten

```
$ cd /home/visiby/计算机组成原理课设/spmv-course-materials
$ git log --date=short --pretty=format:%h  %ad  %s -18

3bed0501  2026-10-31  2026.10.31v3实验记录
b67c6374  2026-10-31  优化方案v3
99b93e19  2026-10-31  2026.10.31v2.2.2实验记录
77bbb9c2  2026-10-31  优化方案v2.2.2
f5f4ddf9  2026-10-30  2026.10.30v2.2.1实验记录
f6e247a3  2026-10-30  优化方案v2.2.1设计
2927723c  2026-10-27  2026.10.27v2.2实验记录
645b2928  2026-10-27  优化方案v2.2设计
5d691bd2  2026-10-26  2026.10.26v2.1实验结果
35609080  2026-10-26  优化方案v2.1设计
3bf5ff5f  2026-10-26  2026.10.26v2实验记录
aaea7f43  2026-10-25  v1缺陷排查与v2设计
30e5c021  2026-10-25  2026.10.25v1实验记录
aa90c775  2026-10-25  2026.10.25优化方案v1
e97d84aa  2026-10-25  2026.10.25原始项目的baseline
2aa47632  2026-10-24  2026.10.24日志更新和项目说明
```

图 7-2 材料仓库设计与实验记录提交历史

7.3 上游 PR 提交记录

图 7-3 为原作者仓库 PR 页面的实际截图，标题为 Final optimized version，目标为 jamtrott:master，来源为 visibychen:optimization/v2。页面显示 2 个提交，与已验收 v2 及最终收尾提交对应。当前状态为 Open，尚无合并或审阅通过的证据。

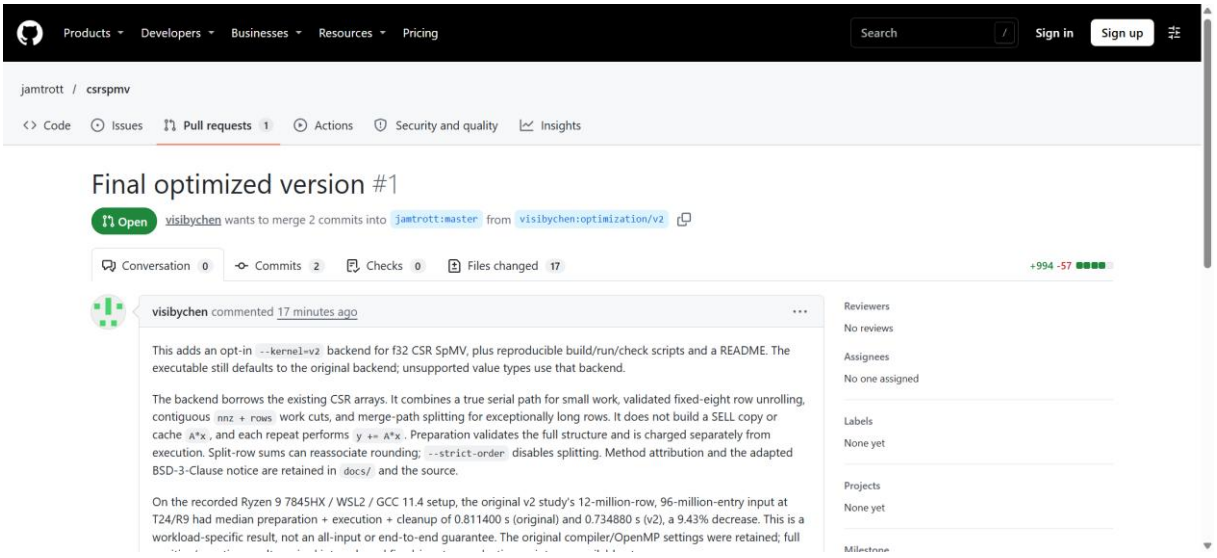


图 7-3 向原作者仓库提交的 PR 页面

上游 PR #1

核心最终提交 a566fd1

材料仓库提交历史